

HireNova

Production Database Schema · PostgreSQL + Prisma ORM

Models	Enums	Modules	AI-Ready
20	11	8	Yes

Enums

Enum Name	Values
UserRole	JOB_SEEKER · RECRUITER · ADMIN
ExperienceLevel	INTERN · ENTRY · MID · SENIOR · LEAD · EXECUTIVE
JobType	FULL_TIME · PART_TIME · CONTRACT · FREELANCE · INTERNSHIP
LocationType	REMOTE · ONSITE · HYBRID
JobStatus	DRAFT · ACTIVE · PAUSED · CLOSED · EXPIRED
ApplicationStatus	PENDING · VIEWED · SHORTLISTED · INTERVIEW_SCHEDULED · OFFERED · HIRED · REJECTED · WITHDRAWN
InterviewType	VIDEO · PHONE · IN_PERSON · TECHNICAL · PANEL
InterviewStatus	SCHEDULED · COMPLETED · CANCELLED · NO_SHOW
ConnectionStatus	PENDING · ACCEPTED · REJECTED · BLOCKED
SkillLevel	BEGINNER · INTERMEDIATE · ADVANCED · EXPERT
CompanySize	STARTUP · SMALL · MEDIUM · LARGE · ENTERPRISE
AdminLevel	SUPPORT · MODERATOR · SUPER_ADMIN
NotificationType	APPLICATION_UPDATE · INTERVIEW_SCHEDULED · JOB_ALERT · CONNECTION_REQUEST · CONNECTION_ACCEPTED

Module 1 — User System

User

Central identity table. Every authenticated entity is a row here. Role-specific data lives in extension profiles (1-to-1 optional).

```
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  phone       String?   @unique
  passwordHash String
  role        UserRole
  firstName   String
  lastName    String
  avatarUrl   String?
  location    String?
  isActive    Boolean   @default(true)
  isVerified  Boolean   @default(false)
  lastSignInAt DateTime?
  deletedAt   DateTime? // soft delete
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

Indexes: email · role · deletedAt

UserSession

Stores refresh tokens for Better Auth / custom JWT. Supports multi-device logout.

```
model UserSession {
  id          String    @id @default(cuid())
  userId      String
  token       String    @unique
  expiresAt   DateTime
  ipAddress   String?
  userAgent   String?
  createdAt   DateTime @default(now())
}
```

Indexes: userId · token

JobSeekerProfile

1-to-1 extension of User for job seekers. Stores career preferences and resume link.

```
model JobSeekerProfile {
  id          String      @id @default(cuid())
  userId      String      @unique
  headline    String?     @db.VarChar(160)
  bio         String?     @db.Text
  resumeUrl   String?
  portfolioUrl String?
  linkedinUrl String?
  githubUrl   String?
  experienceLevel ExperienceLevel @default(ENTRY)
  openToWork  Boolean      @default(true)
  expectedSalary Int?
  preferredTypes JobType[]
  preferredLocs LocationType[]
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}
```

Indexes: `experienceLevel` · `openToWork`

RecruiterProfile

1-to-1 extension for recruiters. Linked to a Company.

```
model RecruiterProfile {
  id      String  @id @default(cuid())
  userId  String  @unique
  companyId String
  title   String?
  bio     String? @db.Text
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Indexes: `companyId`

AdminProfile

1-to-1 extension for admins with role level.

```
model AdminProfile {
  id      String  @id @default(cuid())
  userId  String  @unique
  level   AdminLevel @default(MODERATOR)
  createdAt DateTime @default(now())
}
```

WorkExperience

Normalized work history rows per job seeker (not stored as JSON).

```
model WorkExperience {
  id          String    @id @default(cuid())
  profileId   String
  company     String
  title       String
  description  String?   @db.Text
  startDate   DateTime
  endDate     DateTime?
  isCurrent   Boolean    @default(false)
  createdAt   DateTime   @default(now())
}
```

Indexes: profileId

Education

Normalized education rows per job seeker.

```
model Education {
  id          String    @id @default(cuid())
  profileId   String
  institution  String
  degree      String
  field       String?
  startYear   Int
  endYear     Int?
  createdAt   DateTime   @default(now())
}
```

Indexes: profileId

Module 2 — Company & Job System

Company

Represents employer organisations. Multiple recruiters can belong to one company.

```
model Company {  
  id          String      @id @default(cuid())  
  name        String  
  slug        String      @unique  
  email       String?     @unique  
  industry    String  
  website     String?  
  headquarters String?  
  size        CompanySize @default(SMALL)  
  logoUrl     String?  
  description  String?     @db.Text  
  isVerified  Boolean      @default(false)  
  deletedAt   DateTime?  
  createdAt   DateTime     @default(now())  
  updatedAt   DateTime     @updatedAt  
}
```

Indexes: slug · industry

JobCategory

Flat taxonomy for job roles (Engineering, Design, Marketing...).

```
model JobCategory {  
  id   String  @id @default(cuid())  
  name String   @unique  
  slug String   @unique  
  icon String?  
}
```

Indexes: slug

Job

Core job posting. Soft-deletable. Denormalised view/application counters for dashboards.

```
model Job {
  id          String          @id @default(cuid())
  recruiterId String
  companyId   String
  categoryId  String
  title       String
  description  String          @db.Text
  requirements String?         @db.Text
  benefits    String?         @db.Text
  jobType     JobType
  status      JobStatus       @default(DRAFT)
  locationType LocationType
  location    String?
  salaryMin   Int?
  salaryMax   Int?
  salaryCurrency String?      @default("USD")
  experienceLevel ExperienceLevel
  applicationCount Int        @default(0)
  viewCount   Int            @default(0)
  expiresAt   DateTime?
  deletedAt   DateTime?
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt
}
```

Indexes: `status · companyId · categoryId · recruiterId · (jobType, locationType) · (salaryMin, salaryMax) · deletedAt · createdAt`
`DESC`

Module 3 — Skill System

Skill

Master skill catalogue. Referenced by both jobs and candidates.

```
model Skill {
  id      String  @id @default(cuid())
  name    String  @unique
  slug    String  @unique
  category String?
}
```

Indexes: `slug`

JobSkill

M-N join: Job <-> Skill. Carries a required flag for must-have vs nice-to-have.

```
model JobSkill {
  jobId   String
  skillId String
  required Boolean @default(true)

  @@id([jobId, skillId])
}
```

CandidateSkill

M-N join: JobSeekerProfile <-> Skill. Stores proficiency level and years of experience.

```
model CandidateSkill {
  profileId String
  skillId   String
  level     SkillLevel @default(INTERMEDIATE)
  yearsExp  Int         @default(0)

  @@id([profileId, skillId])
}
```

Indexes: skillId

Module 4 — Application System

Application

Rich join between a job seeker and a job. Unique per (job, seeker) pair. Caches AI match score at apply time.

```
model Application {
  id              String           @id @default(cuid())
  jobId           String
  seekerId        String
  status          ApplicationStatus @default(PENDING)
  coverLetter     String?          @db.Text
  resumeSnapshotUrl String?
  matchScore      Float?           // cached at apply time
  recruiterNotes  String?          @db.Text
  appliedAt       DateTime         @default(now())
  updatedAt       DateTime         @updatedAt

  @@unique([jobId, seekerId])
}
```

Indexes: jobId · seekerId · status · appliedAt DESC

ApplicationStatusLog

Immutable audit trail of every status transition. Required for compliance and dispute resolution.

```
model ApplicationStatusLog {
  id              String           @id @default(cuid())
  applicationId   String
  fromStatus      ApplicationStatus?
  toStatus        ApplicationStatus
  changedBy       String           // userId of actor
  note            String?
  createdAt       DateTime         @default(now())
}
```

Indexes: applicationId

Interview

Scheduled interviews linked to an application. Stores video link, notes, and feedback.

```
model Interview {
  id          String          @id @default(cuid())
  applicationId String
  recruiterId String
  scheduledAt DateTime
  durationMins Int             @default(60)
  interviewType InterviewType @default(VIDEO)
  meetLink    String?
  notes       String?         @db.Text
  feedback    String?         @db.Text
  status      InterviewStatus @default(SCHEDULED)
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt
}
```

Indexes: applicationId · recruiterId · scheduledAt

Module 5 — AI & Matching System

MatchScore

Precomputed AI match between a candidate and a job. Recalculated by background worker on profile/job changes. breakdown JSON stores per-dimension scores.

```
model MatchScore {
  id          String  @id @default(cuid())
  jobId       String
  profileId   String
  score       Float   // 0.0 - 1.0
  breakdown   Json    @default("{}") // {skillMatch, expMatch, ...}
  modelVersion String @default("v1")
  computedAt  DateTime @default(now())

  @@unique([jobId, profileId])
}
```

Indexes: (profileId, score DESC) · (jobId, score DESC)

RecommendationLog

Analytics log for the recommendation engine. clicked and applied booleans form the feedback loop for model retraining.

```
model RecommendationLog {
  id      String   @id @default(cuid())
  userId  String
  jobId   String
  score   Float
  reason  String?  // e.g. skill_match, similar_role, trending
  clicked Boolean  @default(false)
  applied Boolean  @default(false)
  shownAt DateTime @default(now())
}
```

Indexes: (userId, shownAt DESC) · jobId

Module 6 — Social / Network System

SavedJob

Job bookmarks per user. Unique per (user, job) pair.

```
model SavedJob {
  id      String   @id @default(cuid())
  userId  String
  jobId   String
  savedAt DateTime @default(now())

  @@unique([userId, jobId])
}
```

Indexes: userId

Connection

Self-referential social graph between users. Unique per (sender, receiver) pair.

```
model Connection {
  id          String           @id @default(cuid())
  senderId    String
  receiverId  String
  status      ConnectionStatus @default(PENDING)
  message     String?          @db.VarChar(300)
  createdAt   DateTime         @default(now())
  updatedAt   DateTime         @updatedAt

  @@unique([senderId, receiverId])
}
```

Indexes: (receiverId, status) · (senderId, status)

Module 7 — Notification System

Notification

User-scoped notifications. `referenceId` + `referenceType` allow deep linking to the related entity.

```
model Notification {
  id          String          @id @default(cuid())
  userId      String
  type        NotificationType
  title       String
  message     String          @db.Text
  referenceId String?         // e.g. jobId, applicationId
  referenceType String?       // "job" | "application" | "connection"
  isRead      Boolean         @default(false)
  readAt     DateTime?
  createdAt   DateTime        @default(now())
}
```

Indexes: `(userId, isRead)` · `(userId, createdAt DESC)`

Module 8 — Admin / Moderation System

ModerationLog

Immutable audit trail of every admin action taken on users, jobs, or companies.

```
model ModerationLog {
  id          String  @id @default(cuid())
  adminId     String
  targetId    String  // ID of entity acted on
  targetType  String  // "user" | "job" | "company"
  action      String  // "suspend" | "delete" | "warn" | "verify"
  reason      String? @db.Text
  metadata    Json?   @default("{}")
  createdAt   DateTime @default(now())
}
```

Indexes: `adminId` · `(targetId, targetType)` · `createdAt DESC`

PlatformSnapshot

Daily KPI snapshots populated by cron job. Never query live aggregates for dashboards — use this table instead.

```
model PlatformSnapshot {
  id          String  @id @default(cuid())
  date        DateTime @unique @db.Date
  totalUsers  Int      @default(0)
  activeJobs  Int      @default(0)
  applicationsToday Int  @default(0)
  hiredToday  Int      @default(0)
  avgMatchScore Float?
  createdAt   DateTime @default(now())
}
```

Indexes: `date DESC`

Relationship Summary

Relationship	Type	Tables
User → JobSeekerProfile	1-to-1 optional	Role-based profile extension
User → RecruiterProfile	1-to-1 optional	Role-based profile extension
RecruiterProfile → Company	Many-to-1	Multiple recruiters per company
Job → Skills	M-to-N (via JobSkill)	Carries required flag
Candidate → Skills	M-to-N (via CandidateSkill)	Carries level + yearsExp
JobSeeker → Job	M-to-N (via Application)	Rich join with status tracking
Job → MatchScore	M-to-N	AI-computed score per pair
Application → Interview	1-to-Many	Multiple rounds per application
Application → StatusLog	1-to-Many	Immutable audit trail
User → User	Self-referential M-N	Via Connection (sender/receiver)

Scalability & AI Integration Notes

- **Match worker:** Triggered on profile or job changes. Upserts into MatchScore with a breakdown JSON column. The modelVersion field enables A/B testing between scoring models without a migration.
- **Recommendation worker:** Reads MatchScore rankings, writes RecommendationLog per user. The clicked and applied boolean columns form a feedback loop for model retraining. JOIN recommendation_logs to applications to measure recommendation-to-hire conversion rate.
- **Soft deletes:** User and Job tables use deletedAt instead of hard deletes — preserves application and audit history for compliance.
- **Denormalised counters:** applicationCount and viewCount on Job are updated via background jobs or DB triggers, avoiding expensive COUNT queries on hot dashboard pages.
- **PlatformSnapshot:** Daily cron-populated KPI table. Admin dashboards query this table — never live aggregations — for O(1) analytics at any scale.
- **Session table:** UserSession supports multi-device logout and is compatible with Better Auth, NextAuth, and custom JWT implementations.
- **Array columns:** preferredTypes and preferredLocs on JobSeekerProfile use PostgreSQL native arrays — filterable with @> operators without a join table for simple preference queries.
- **Full-text search:** Enable Prisma previewFeature fullTextSearch and add a GIN index on Job(title, description) for fast keyword search without an external search engine.